DG Interns Hub

Cyber Security Internship Program

Week 8: Advanced VAPT

Attack Chain and Full Exploitation Report

Target Application: OWASP Juice Shop

Prepared by

Mah Noor

Intern ID: DG/AUGUST/CYBER/077

October 14, 2026

1. Introduction

This report documents an advanced Vulnerability Assessment and Penetration Testing (VAPT) exercise carried out as part of Week 8 of the DG Interns Hub Cyber Security Internship. Unlike earlier assignments that focused on finding isolated vulnerabilities, this exercise required chaining several individually small weaknesses together to reach a meaningful outcome, in this case full administrative access, exposure of other customers personal data, and manipulation of order values.

The testing was carried out against OWASP Juice Shop, an intentionally vulnerable web application that is widely used for training and research purposes. All testing was performed in a local, isolated environment running inside Docker on the testers own machine, with no production systems, real users, or real data involved at any point.

Burp Suite Community Edition was used throughout as the primary interception and testing tool, alongside Mozilla Firefox configured to route its traffic through the Burp proxy.

2. Target and Scope

Target application: OWASP Juice Shop, running locally via Docker at http://localhost:3000.

Testing tools: Burp Suite Community Edition (Proxy, Repeater, HTTP history) and Mozilla Firefox.

Scope of this engagement:

- Authentication and login mechanisms
- Administrative interface and access control
- REST and API endpoints handling user and order data (Users API, Basket API)
- Basket and checkout logic, including quantity and pricing handling
- Session and token handling across login and logout

Out of scope: denial of service testing, testing against any instance of Juice Shop other than the locally hosted container, and any automated mass scanning. The engagement deliberately favoured manual, logic driven testing over basic automated scans, in line with the assignment requirements.

3. Attack Surface Mapping

Before any exploitation was attempted, the application was reviewed manually to identify entry points and understand how its pages and APIs connect to each other. The home page lists all products and exposes a search function, a login and registration flow, and a basket icon leading to checkout.

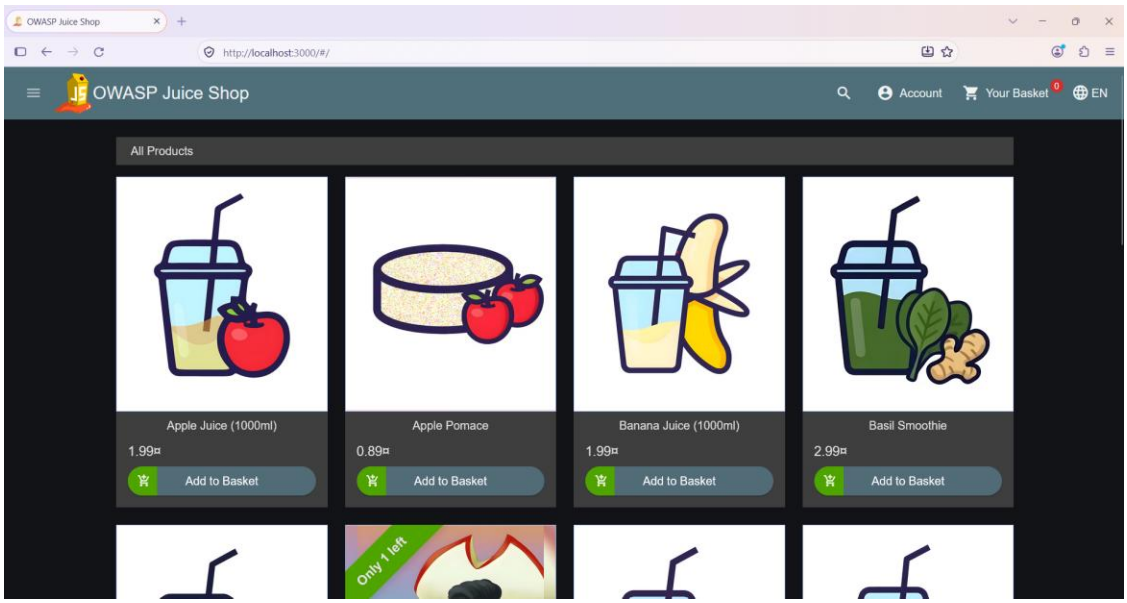


Figure 1. OWASP Juice Shop home page, the main entry point and starting surface for testing.

The hidden Score Board page, which Juice Shop deliberately does not link to from the navigation menu, was located and used to track which categories of vulnerability the application contains. This page also doubles as a running scoreboard of challenges solved during the engagement.

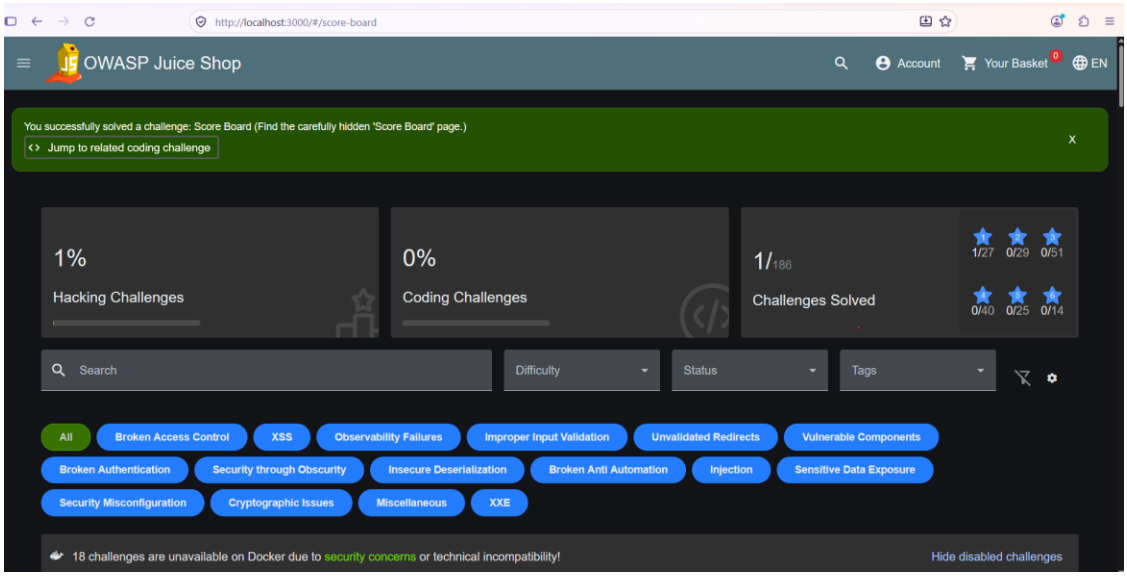


Figure 2. Score board showing available challenge categories including Broken Access Control, Injection, and Broken Authentication.

All traffic between Firefox and the application was routed through Burp Suite so that every request and response could be observed and later replayed. This traffic capture formed the basis of every attack step that follows.

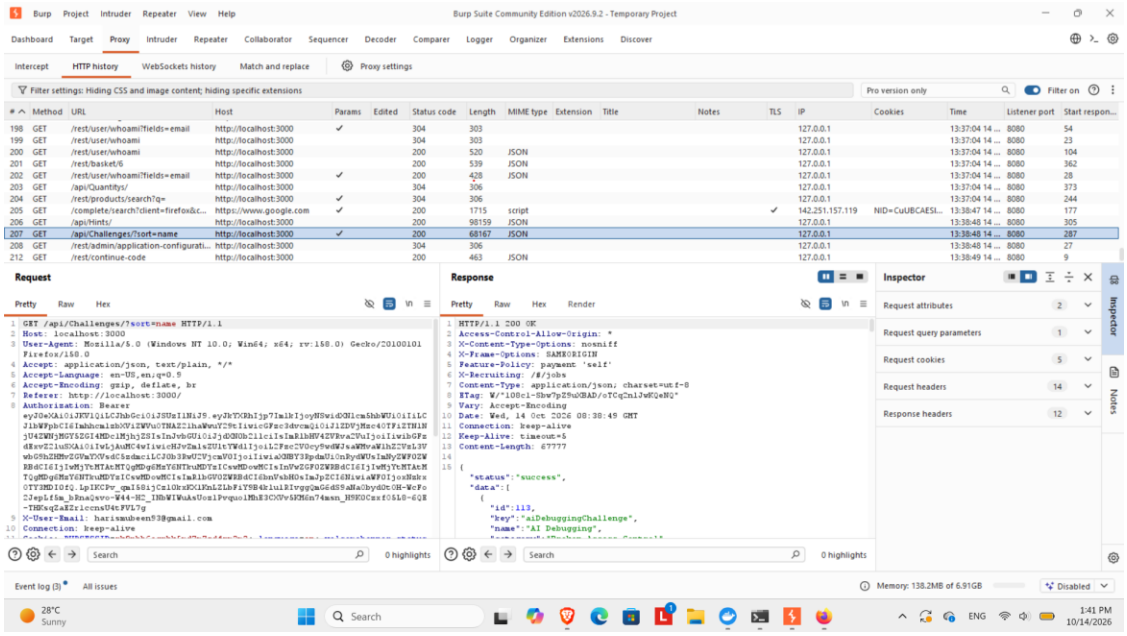


Figure 3. Burp Suite Proxy HTTP history showing captured requests to the application, including API calls carrying the session token.

From this mapping, three categories of entry point were identified as the most promising for an attack chain: the login form, which handles authentication directly against the database; the REST and API endpoints that return data by object ID (Users, Basket, Addresses); and the basket and checkout flow, which accepts client supplied quantity and pricing related values.

4. Attack Chain Explanation

The following five steps were carried out in sequence and build on one another. Each step on its own would be a limited finding, but combined they represent a realistic path an attacker could take from an anonymous visitor to full account takeover, data exposure, and manipulation of orders.

Step 1: Authentication Bypass via SQL Injection

The login form was tested with a classic SQL injection payload in the email field, using a blank or arbitrary password. The payload entered was:

```
' or 1=1--
```

This payload is designed to manipulate the underlying SQL query so that the condition always evaluates to true, bypassing the password check entirely and returning the first matching user in the database, which in Juice Shop is the administrator account.

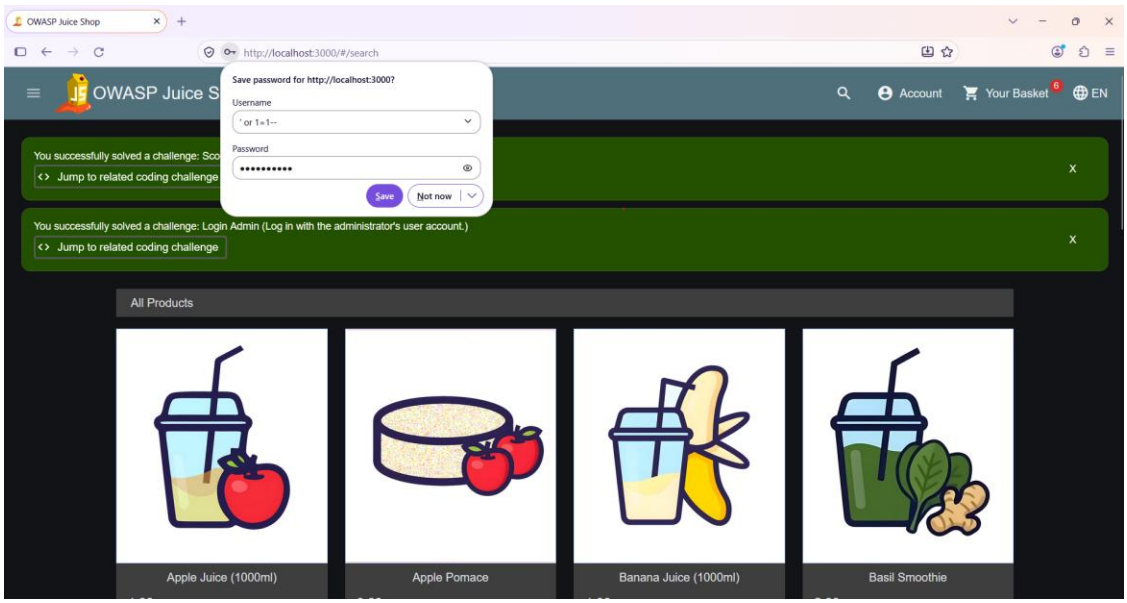


Figure 4. The SQL injection payload entered in the login email field, with the browser offering to save it as a username, confirming the exact string submitted.

The login succeeded without ever knowing the administrators real password. The application itself confirmed this as a successful exploitation by awarding the Login Admin challenge, along with the previously found Score Board challenge.

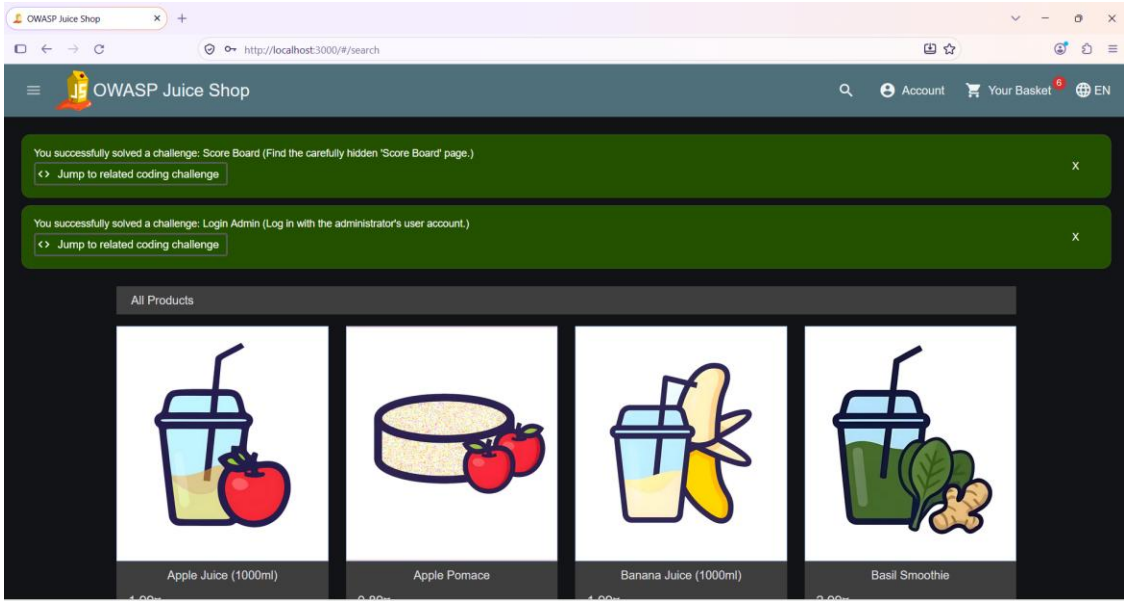


Figure 5. Application confirmation that the Score Board and Login Admin challenges were both solved, verifying the authentication bypass.

Impact of this step: an attacker gains full administrative session access without any valid credentials, which is the foundation for every subsequent step in this chain.

Step 2: Broken Access Control, Admin Panel Exposure

With an authenticated admin session now active, the hidden /#/administration route was accessed directly through the browser address bar. This route is not linked anywhere in the normal navigation menu for non admin users, but the application performed no additional server side check before rendering it once the admin token was present.

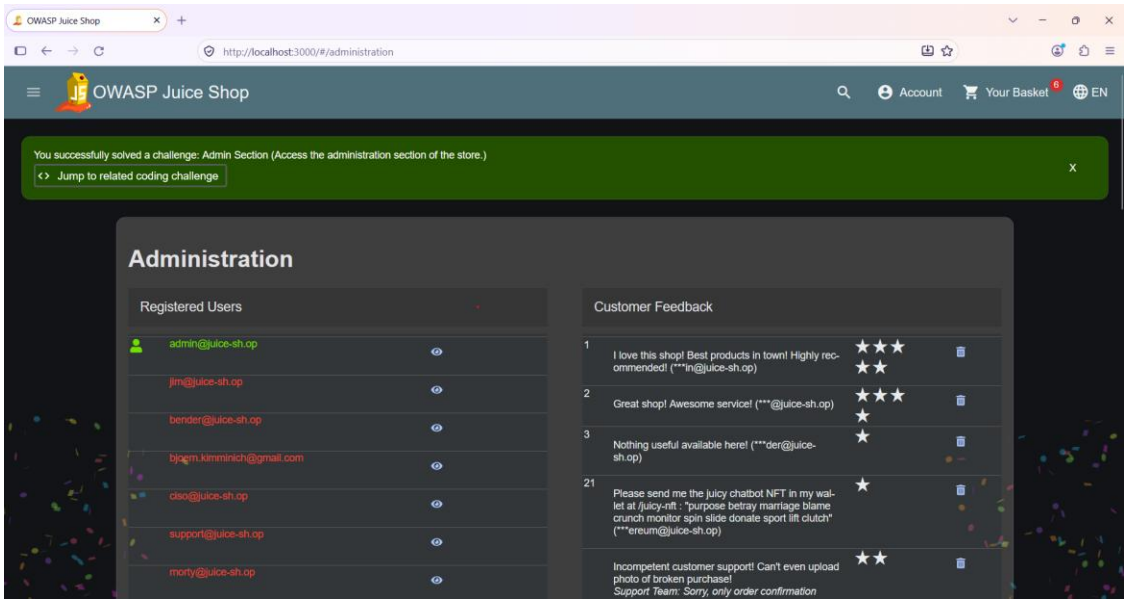


Figure 6. Administration panel fully accessible, listing every registered user account and customer feedback, with the Admin Section challenge marked as solved.

From within this panel, individual user records could be opened to reveal personally identifiable information that should not be visible to another account, including the real email address behind an administrator profile.

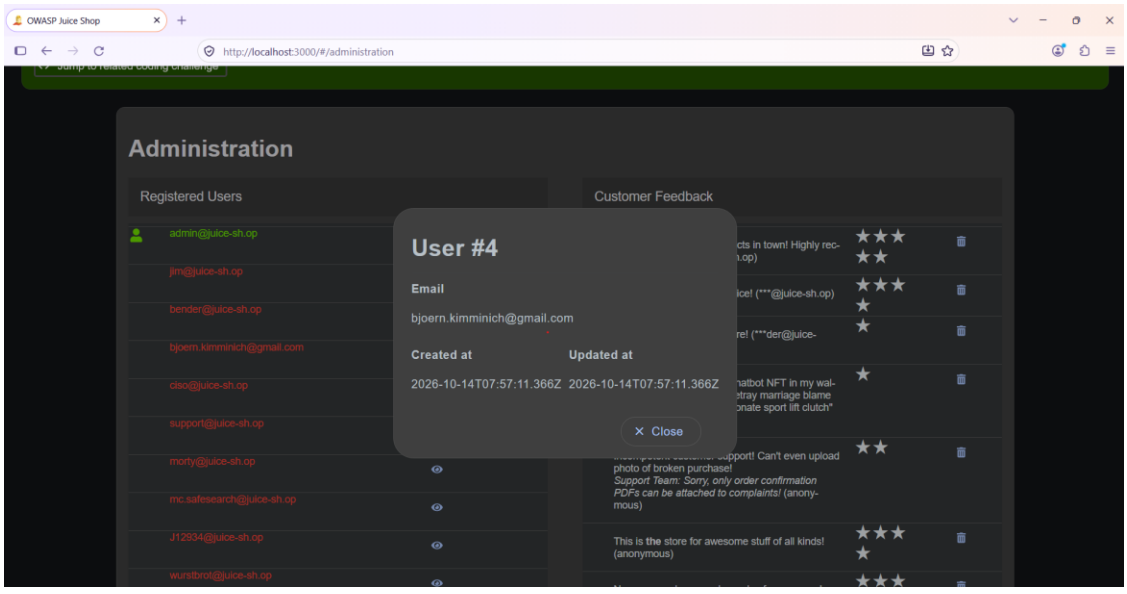


Figure 7. User record opened from the administration panel, exposing the email address, creation date, and update date tied to that account.

Impact of this step: an attacker with a bypassed login can reach a fully functional administrative interface, view every customer record, and read and delete feedback left by other users, none of which should be reachable without a dedicated administrative authorization check.

Step 3: Insecure Direct Object Reference (IDOR)

The applications REST and API endpoints were tested by directly editing the numeric object ID in the URL while keeping the same authenticated session. Two separate endpoints were tested this way.

First, the Users API was called directly for a specific user ID:

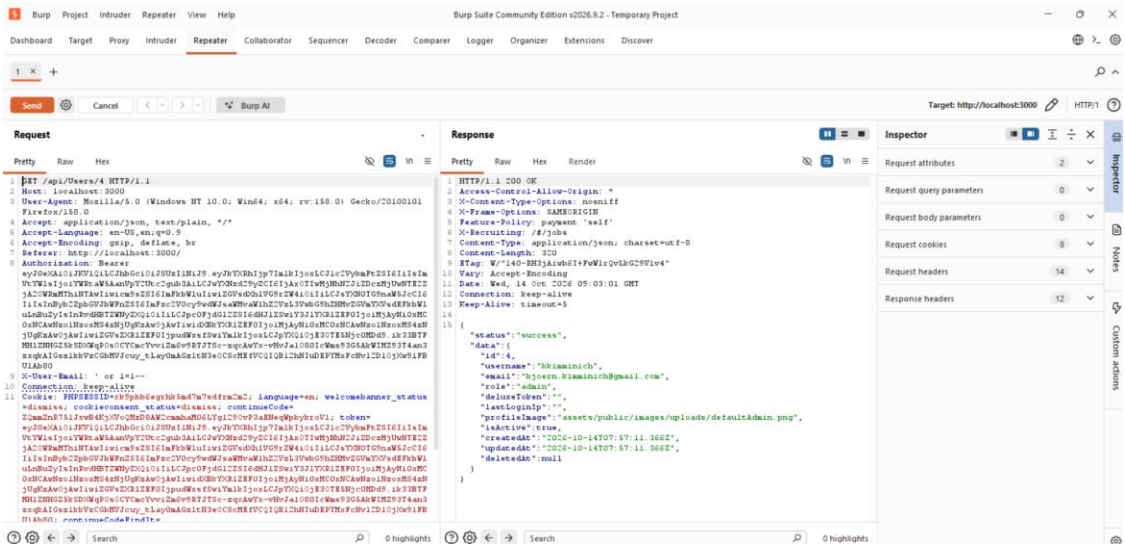


Figure 8. GET request to /api/Users/4 replayed in Burp Repeater, returning the full user record for that account including username, email, role, and profile image path.

Second, and more significantly, the Basket API was tested the same way. By simply changing the numeric ID at the end of the basket URL, baskets belonging to completely different user accounts could be read, each returning a different UserId and a different set of products.

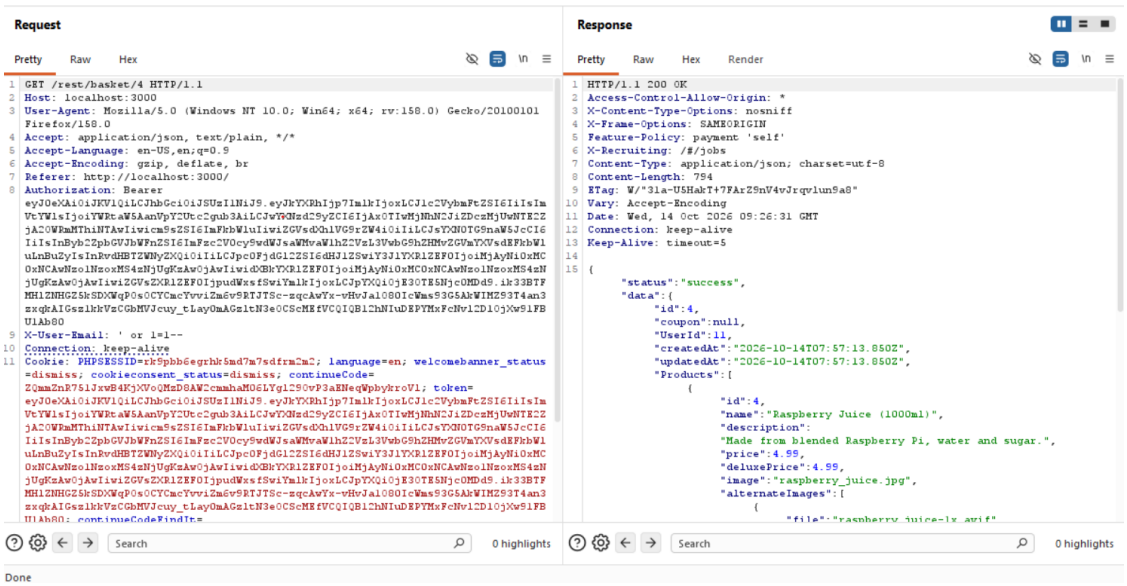


Figure 9. Basket ID 4 returned, belonging to UserId 11, containing a Raspberry Juice order with its price.

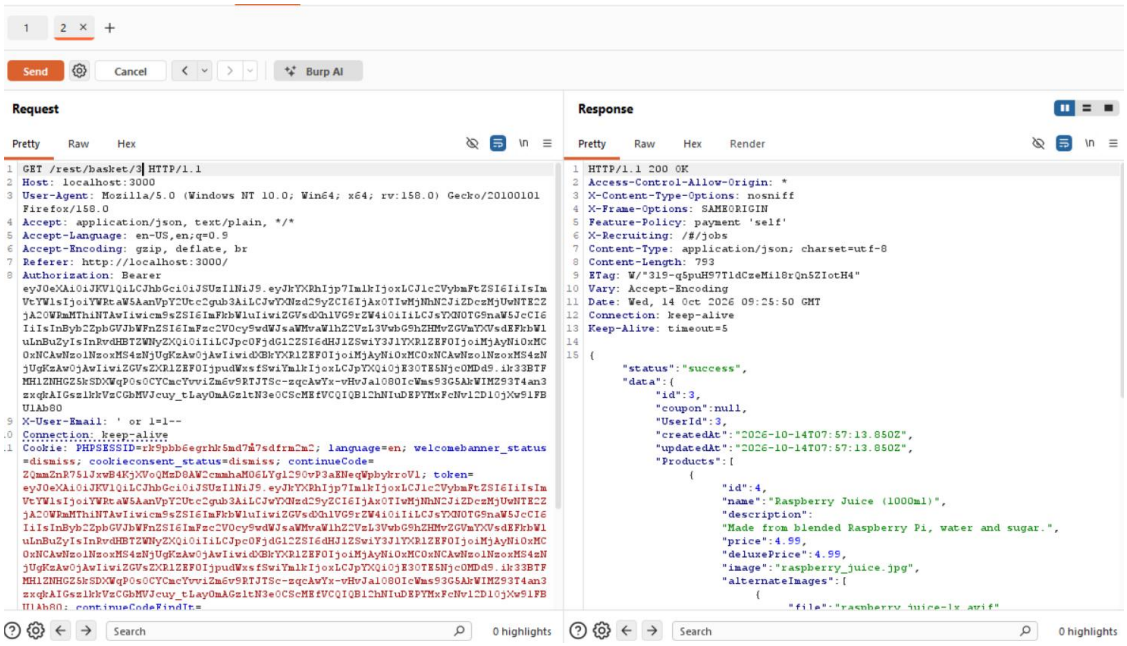


Figure 10. Basket ID 3 returned, belonging to UserId 3, confirming the pattern holds across accounts.

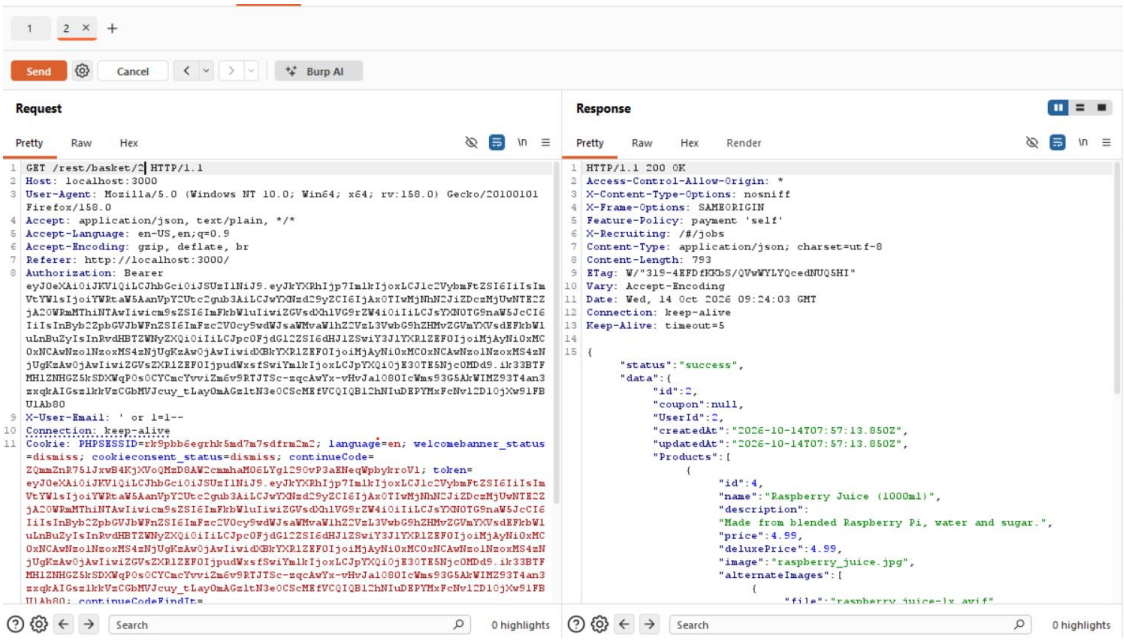


Figure 11. Basket ID 2 returned, belonging to UserId 2.

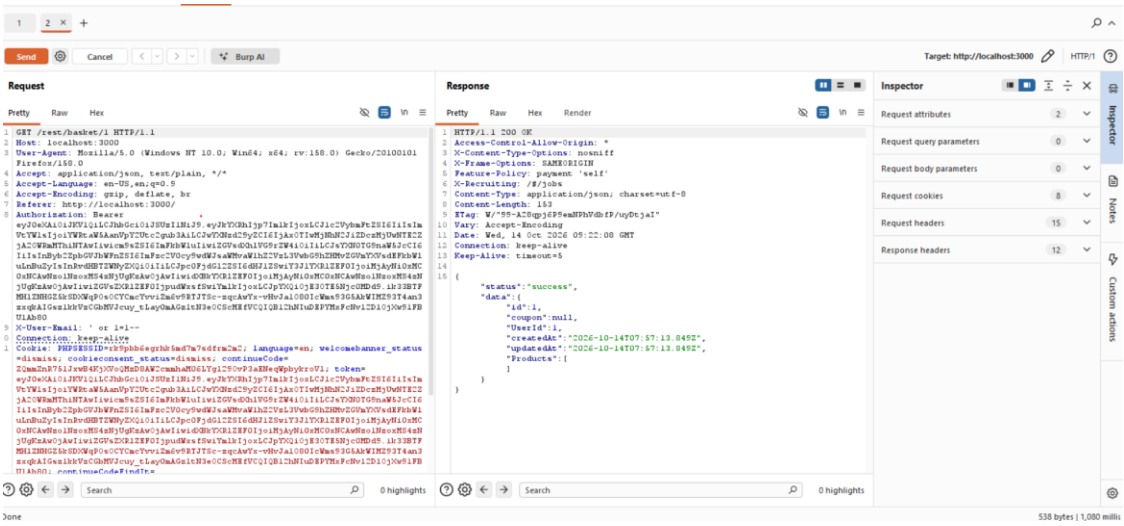


Figure 12. Basket ID 1 returned, belonging to UserId 1, completing the sequence of four different customers whose basket contents were exposed purely by changing one number in the request.

Impact of this step: no ownership check is performed before returning basket or user data for a given ID. Any authenticated user, or in this case an attacker who has already bypassed login once, can enumerate every basket and user ID in the system and read what other customers have ordered, purely by incrementing a number in the URL.

Step 4: Business Logic Flaw, Parameter Tampering on Basket Quantity

The request that adds an item to the basket was intercepted in Burp and sent to Repeater for controlled, repeatable testing. The baseline request simply adds one unit of a product to the basket, as shown below.

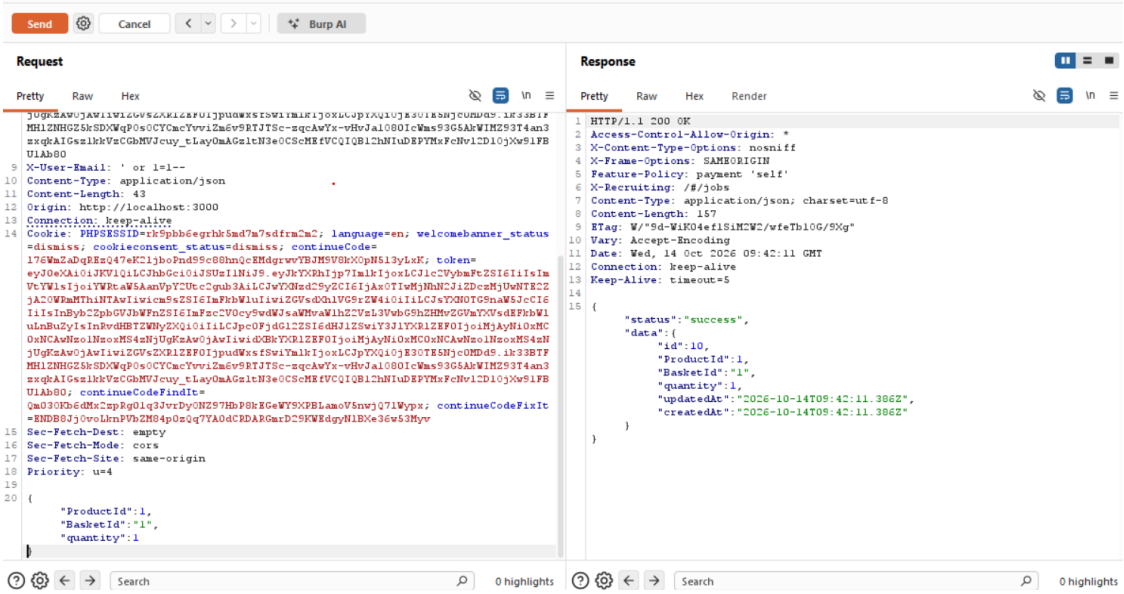


Figure 13. Baseline POST request to /api/BasketItems/ adding one unit of a product, used as the reference point before tampering.

The quantity value in the JSON body was then changed to an arbitrary larger number. The server accepted this without any server side validation or upper limit, confirming that the client is trusted to send a correct quantity.

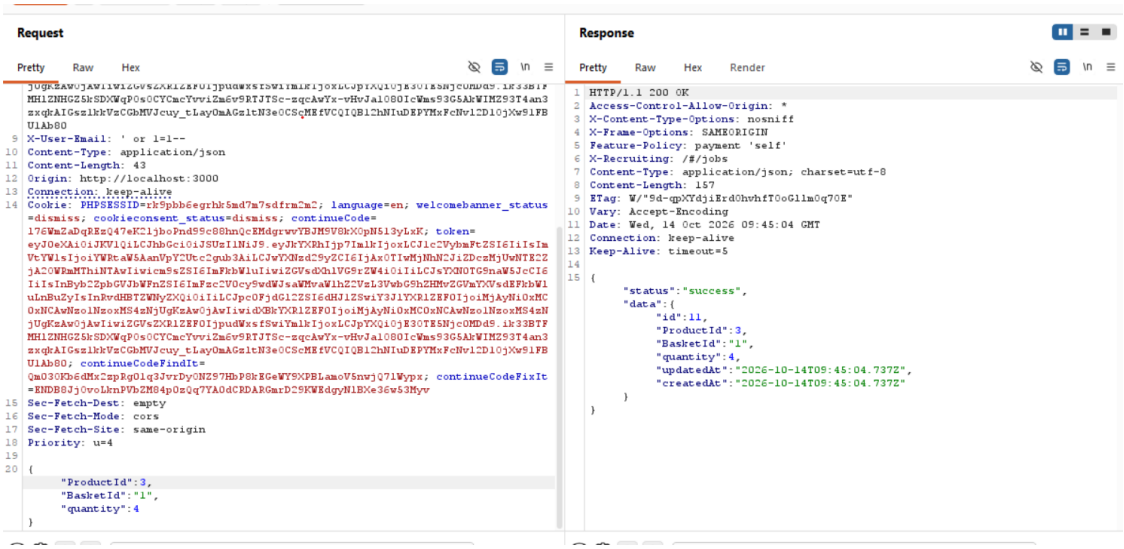


Figure 14. Tampered request with an increased quantity value, accepted by the server with a success response.

The same request was then tested with a negative quantity, which represents a more severe logic flaw because a negative quantity can turn a purchase into a credit against the basket total.

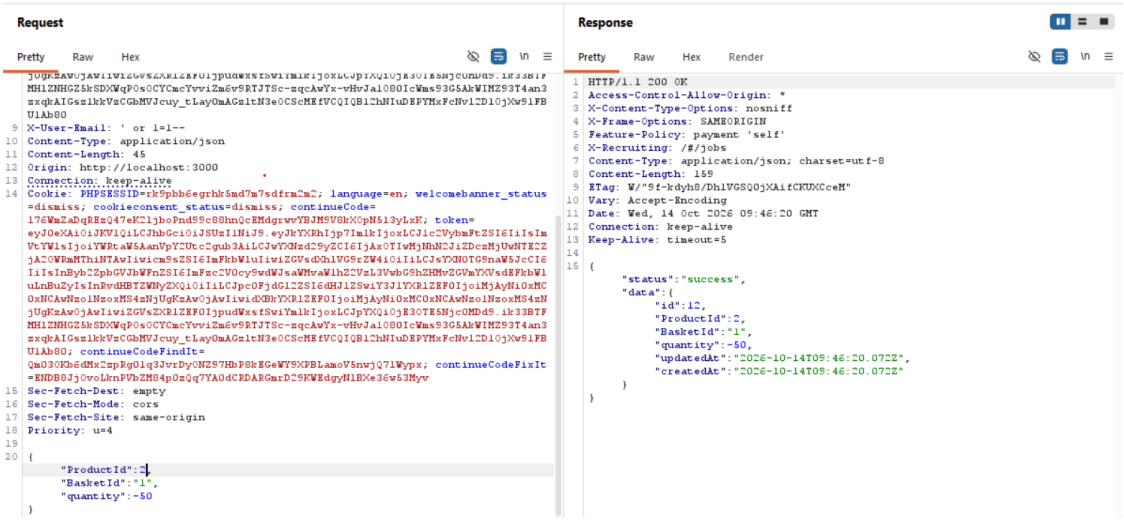


Figure 15. Tampered request with quantity set to negative fifty, again accepted by the server with a success response and no validation error.

The real world effect of this flaw was visible directly in the application interface. After adding items with manipulated quantities, the basket item counter itself turned negative, and the application unintentionally triggered its own Error Handling challenge, confirming that this input was never expected or properly handled by the developers.

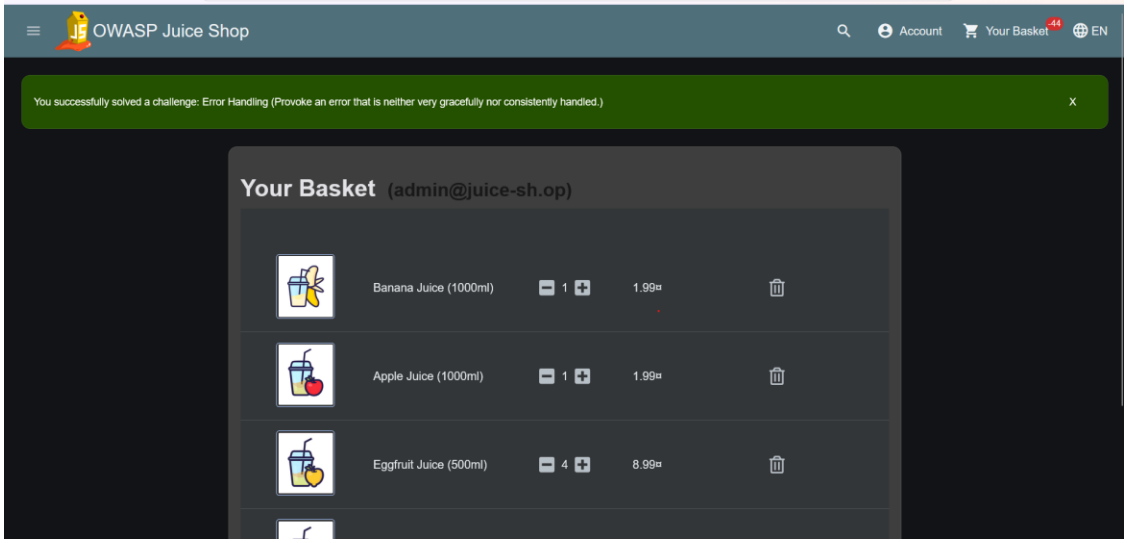


Figure 16. Basket icon showing a negative item count of minus forty four as a direct result of the tampered quantities, alongside the applications own confirmation of the Error Handling challenge being solved.

Impact of this step: an attacker can manipulate order quantities freely, including negative values, which could be abused to distort basket totals, trigger unhandled application errors, or in a real production system potentially manipulate billing in the attackers favour.

Step 5: Weak Session Management, Token Reuse After Logout

A request carrying the authenticated Bearer token was first replayed successfully before logging out, confirming the token was valid and the account was active.

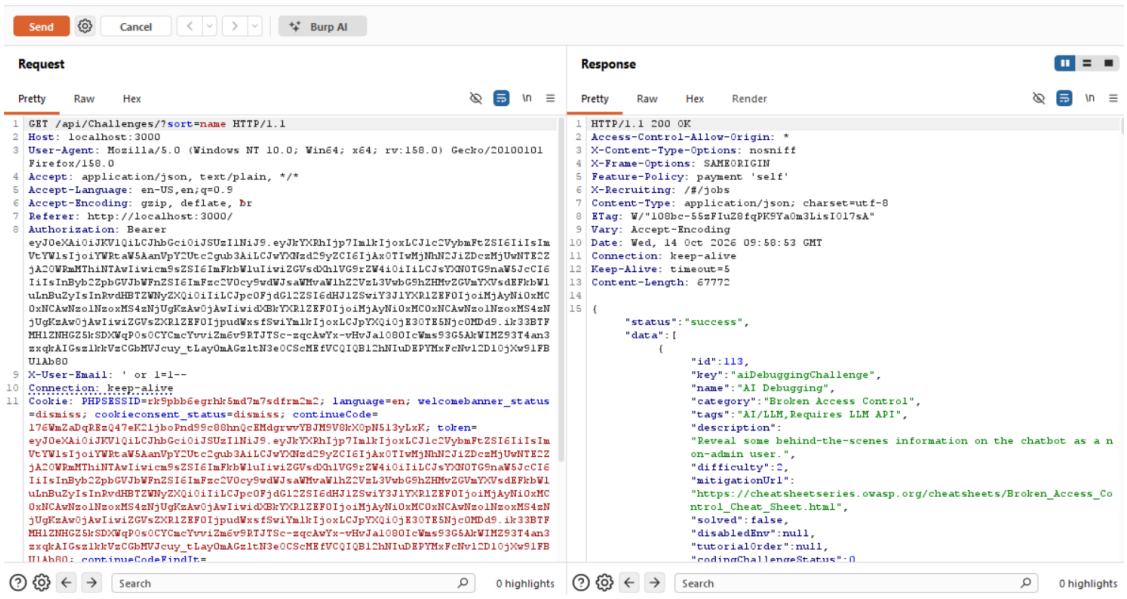


Figure 17. Authenticated request to /api/Challenges/ replayed in Repeater before logout, returning a successful response with full challenge data.

The account was then logged out normally through the application interface. Without changing a single character of the captured request, including the same Authorization header and Bearer token, the exact same request was replayed again in Burp Repeater.

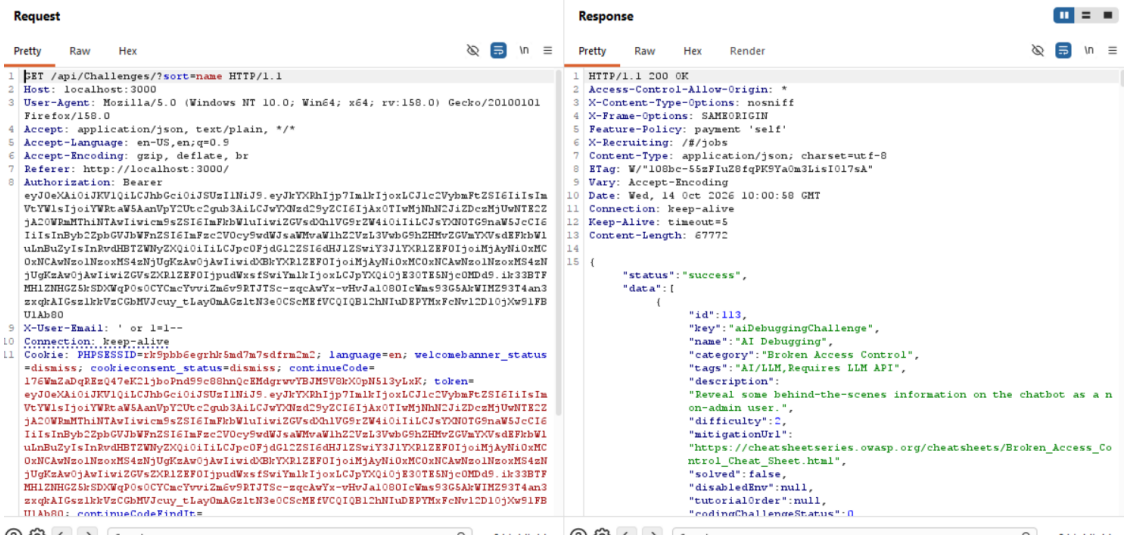


Figure 18. The identical request replayed after logout, still returning a success status and the full data payload, proving the token was never invalidated server side.

Impact of this step: logging out only affects the client side application state, such as clearing local storage in the browser. The token itself remains valid and usable against the API. If this token were ever captured by an attacker, for example through a shared device, a proxy log, or a man in the middle position, logging out would provide no real protection against its continued use.

5. Consolidated Attack Chain

Put together, the five steps above form a single continuous attack path from an anonymous visitor to a fully compromised account with data exposure and manipulated orders:

- Step 1: SQL injection in the login form grants administrator level access without valid credentials.
- Step 2: The admin session is used to reach the hidden administration panel, exposing every registered user and their feedback.
- Step 3: IDOR on the Users and Basket APIs is used to enumerate and read data belonging to other customers.
- Step 4: Parameter tampering on basket quantity manipulates order values, including negative quantities that break the applications own logic.
- Step 5: The session token captured at login remains valid even after logout, meaning the entire access gained in steps one through four persists beyond the point where the application believes the user has logged out.

No single one of these issues requires the others to exist, but together they show how a small authentication weakness can cascade into complete compromise of customer data and application logic when access control, input validation, and session handling are all weak at the same time.

6. Risk and Impact Analysis

The table below summarizes the business impact and severity rating of each vulnerability identified in this engagement, rated using a standard Low, Medium, High, Critical scale based on exploitability and potential damage.

Vulnerability	Business Impact	Severity
Authentication Bypass (SQL Injection in Login)	Attacker logs in as any user, including the administrator, without knowing the real password.	**Critical**
Broken Access Control (Admin Panel Exposure)	Attacker reaches an administrative interface that should only be reachable by admin accounts.	**High**
Insecure Direct Object Reference (Users and Basket APIs)	Attacker reads other customers personal data and order contents by changing an ID in the URL.	**High**
Business Logic Flaw / Parameter Tampering (Basket Quantity)	Attacker sets arbitrary and even negative quantities, breaking pricing logic and basket totals.	**High**
Weak Session Management (Token Reuse After Logout)	A captured token keeps working after the legitimate user has logged out.	**Medium**

The authentication bypass is rated Critical because it requires no prior access and directly grants administrator level control. The access control and IDOR findings are rated High because they expose personal data belonging to every customer in the system with no additional effort once the initial bypass succeeds. The session weakness is rated Medium because it requires the token to already be captured by some other means, but it meaningfully extends the real world impact of the other findings by removing logout as an effective containment step.

7. Remediation Strategy

Fixing this attack chain requires addressing each individual weakness, since a single patch will not close every step at once.

Vulnerability	Recommended Fix
SQL Injection Login Bypass	Use parameterized queries or an ORM with bound parameters for every database call. Never build SQL strings by concatenating user input.
Broken Access Control on Admin Panel	Enforce server side role checks on every administrative route, not just a hidden link on the frontend. A missing menu item is not access control.
IDOR on Users and Basket APIs	Verify on the server that the logged in user actually owns the resource being requested, for every object ID, on every request.
Parameter Tampering on Basket Quantity	Validate quantity and price on the server side with a minimum of one and a maximum sensible limit. Never trust a price or quantity value sent from the client.
Weak Session Management	Maintain a server side token blacklist or short lived tokens with refresh, so that logout actually invalidates the session instead of only clearing it from the browser.

Breaking the Chain Completely

Beyond fixing each vulnerability individually, the chain itself can be broken at its weakest link. If the SQL injection in the login form is fixed first, the attacker never obtains the administrator session that made every later step possible in this engagement. This makes authentication hardening, specifically parameterized queries and proper input validation on the login form, the single highest priority fix, since it removes the entry point the entire chain depends on.

As defense in depth, server side authorization checks on every admin route and every object ID based endpoint should still be implemented regardless of how strong the login mechanism is, since relying on a single control to stop an entire attack chain is itself a risky practice.

8. Additional Observations

During the course of this engagement, two further coding challenges related to the Score Board page were also solved as part of understanding how the application conceals and protects its internal routes. These are included here as supporting evidence of the depth of testing performed, though they fall outside the core attack chain documented above.

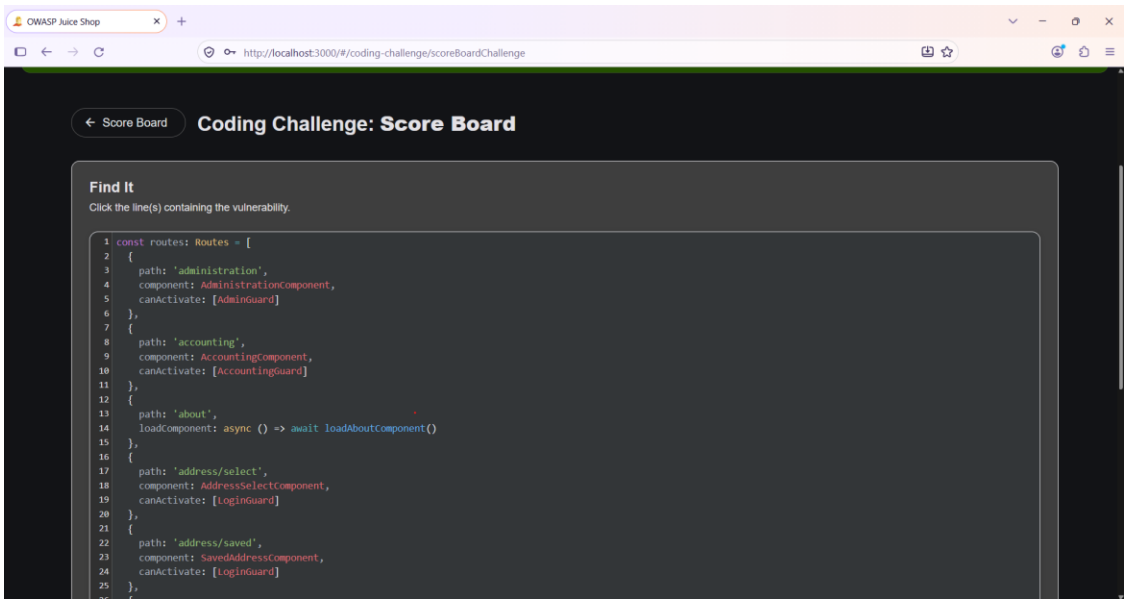


Figure 19. Source code review identifying the route configuration responsible for concealing the Score Board page from the navigation menu.

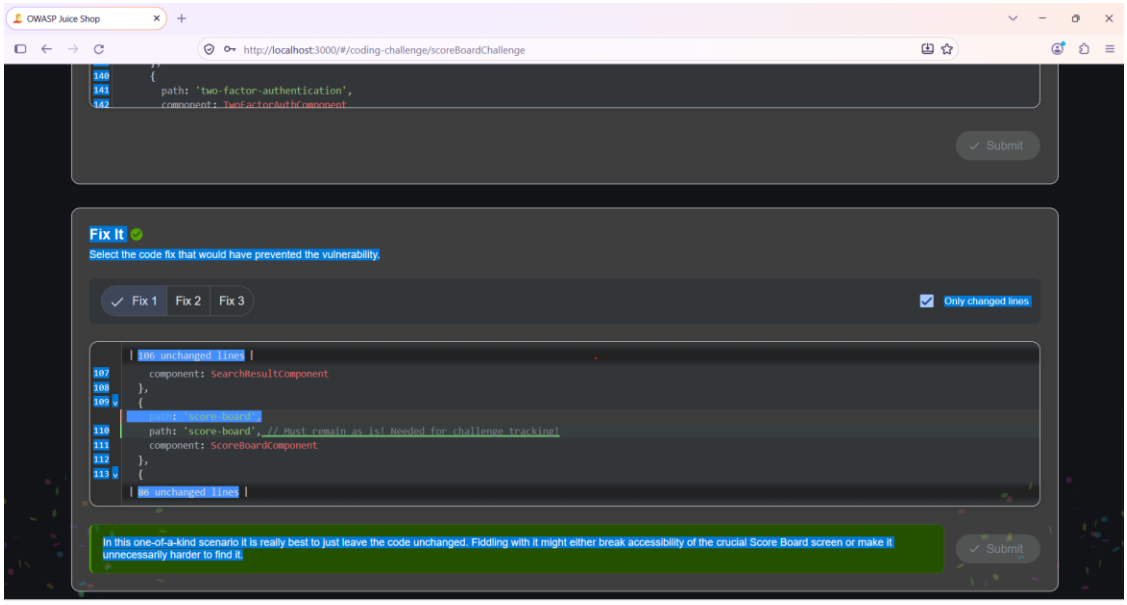


Figure 20. Reviewing the available code fixes for the Score Board route and confirming that the route itself must remain functional for legitimate challenge tracking.

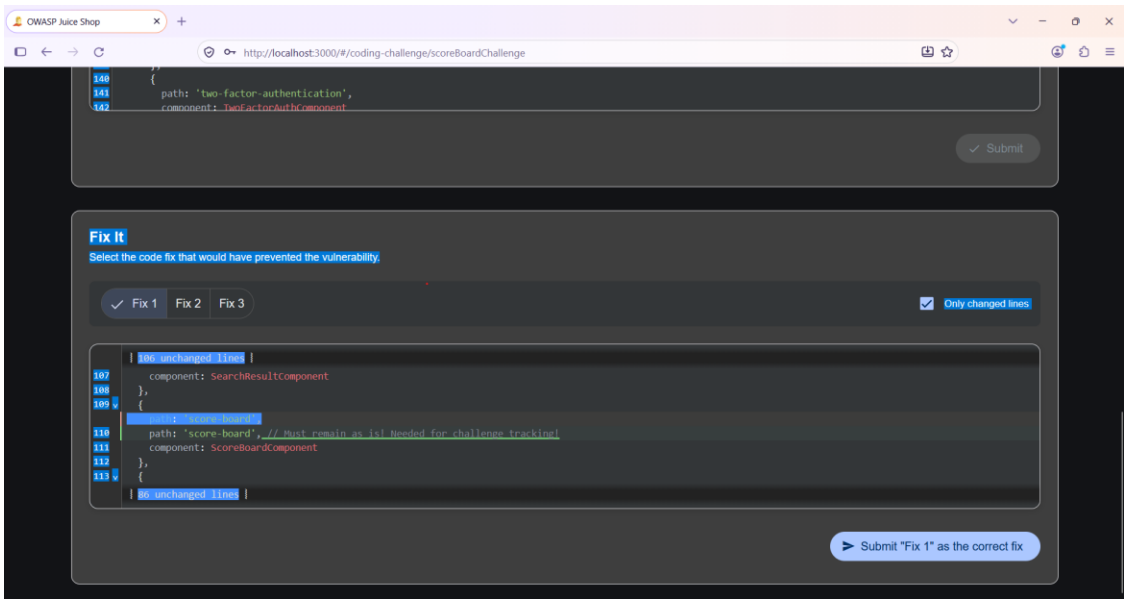


Figure 21. Submitting the correct fix choice for the Score Board coding challenge.

9. Conclusion

This engagement demonstrates that isolated, low severity looking issues such as a missing input validation rule, a hidden but unprotected route, and a token that is never invalidated, can be combined into a serious, end to end compromise of an application. Starting from a single SQL injection payload in a login form, it was possible to reach full administrative access, read every customer record in the system, manipulate basket quantities including into negative values, and retain that access even after a formal logout.

The recommendations in Section 7 of this report, if implemented together rather than individually, would close this attack chain at multiple points and significantly raise the effort required for any similar attack to succeed in the future.